

ADR-010 Teams to Domain associations in the GraphQL Federation

ADR-010 — Relationship Service Association Resolution in the Federated Supergraph

Field	Value
ADR ID	ADR-010
Jira	SPARK-30228
Status	DECIDED
Decision Date	6-1-2026
Decision Makers	Robert.OConnell Ashley.Sparks Scott.McNulty Mitch.Peck Giri.AlkondanSubbiah Saritha.Mathai

Context & Problem Statement

The PLM platform uses a central **Relationship Service** as a graph store for associations between entities. It tracks virtual foreign key relationships across domain boundaries — e.g. which teams are associated with a material, which BOMs reference a fabric spec, which artworks are linked to other artworks, which samples belong to a product.

In the legacy spark-internal-graphql monolith, a shared set of resolver utilities (commonResolvers/Teams.js, commonResolvers/Files.js, commonResolvers/ColorHistoryItems.js) and inline resolver logic handle these lookups via a consistent two-hop pattern:

1. Query the Relationship Service to get associated entity IDs
2. Hydrate the full entity objects from the owning domain service

This pattern is used across many entity types and is not specific to any one domain.

As the platform migrates to a federated GraphQL supergraph, these fields are absent from federated subgraph schemas. A decision is needed on where this resolution logic lives in the federated architecture — and that decision applies broadly, not just to materials.

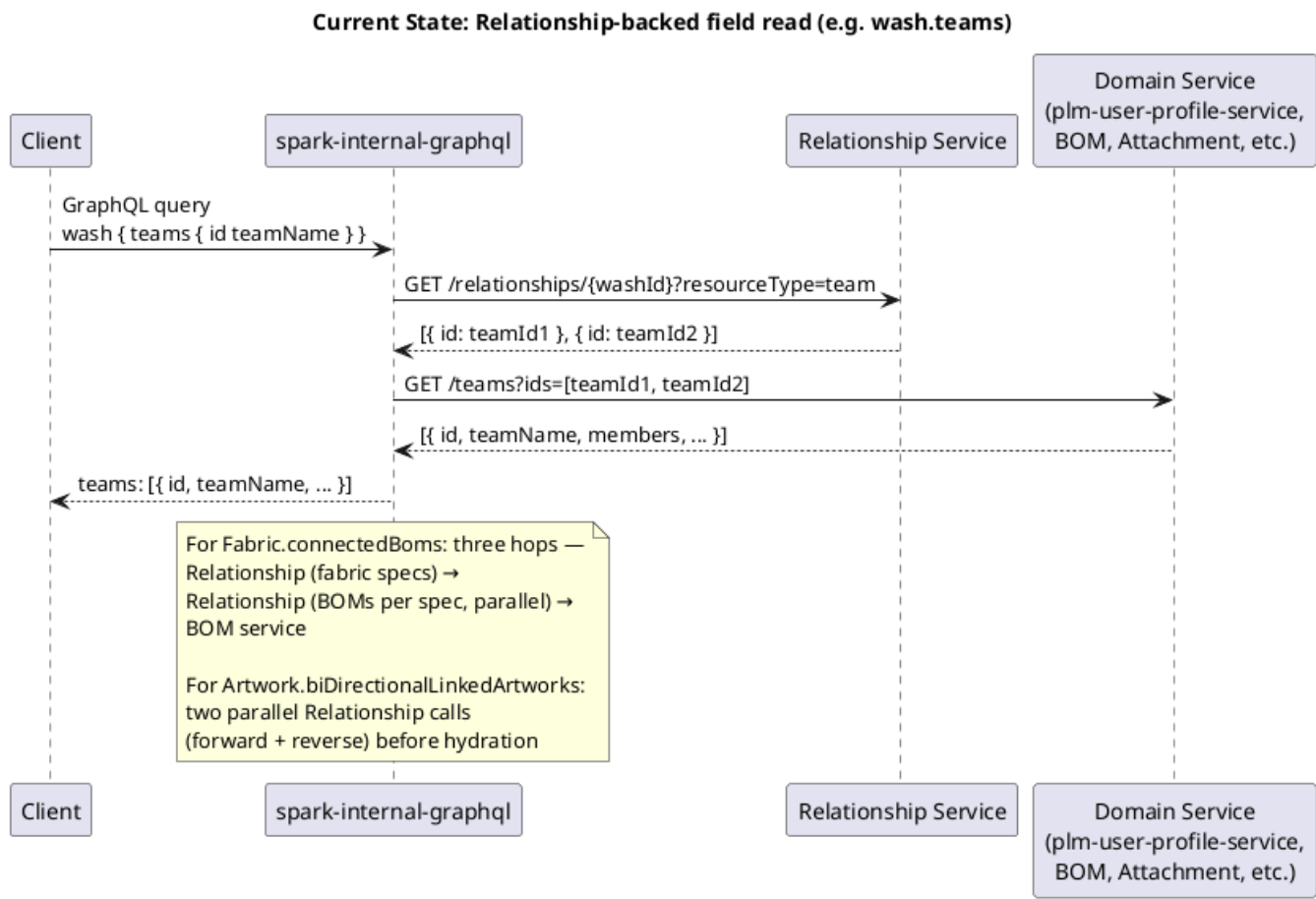
Affected Fields & Domains

The following fields all follow the Relationship Service two-hop pattern and are currently unresolved in the federated supergraph:

Entity Type	Field	Relationship Call	Hydration Target
All 8 material types	teams	getByID type=team	plm-user-profile-service (Team)
All 8 material types	colorHistoryItems	getByID maxDepth=0	Color service
All 8 material types	files / attachments	getByID type=attachments_v3	Attachment service
BaseMaterial (Hub)	connectedBoms	whereUsedSearch bom types	BOM service
BaseMaterial (Hub)	developedBaseMaterials	findRelationships type=material	MaterialHub service
Fabric	connectedBoms	searchByIds then whereUsedSearch per spec (3-hop)	BOM service
Fabric	developedFabrics	getByID type=fabric	Fabric service
Fabric	developedArtworks	getByID type=artworks	Artwork service
Artwork	linkedArtworks	getByID type=artworks	Artwork service
Artwork	biDirectionalLinkedArtworks	getByID + getWhereUsedByID bidirectional	Artwork service
ServiceMaterial	relatedBoms	whereUsedSearch bom types	BOM service
ServiceMaterial	relatedPalettes	whereUsedSearch type=palette	Palette service
ServiceMaterial	samples	whereUsedSearch type=sample	Sample service
Product	attachments	searchByIds attachment types	Attachment service
Product	samples	getByID type=sample	Sample service
Discussion	(helper)	searchByIds	Discussion service

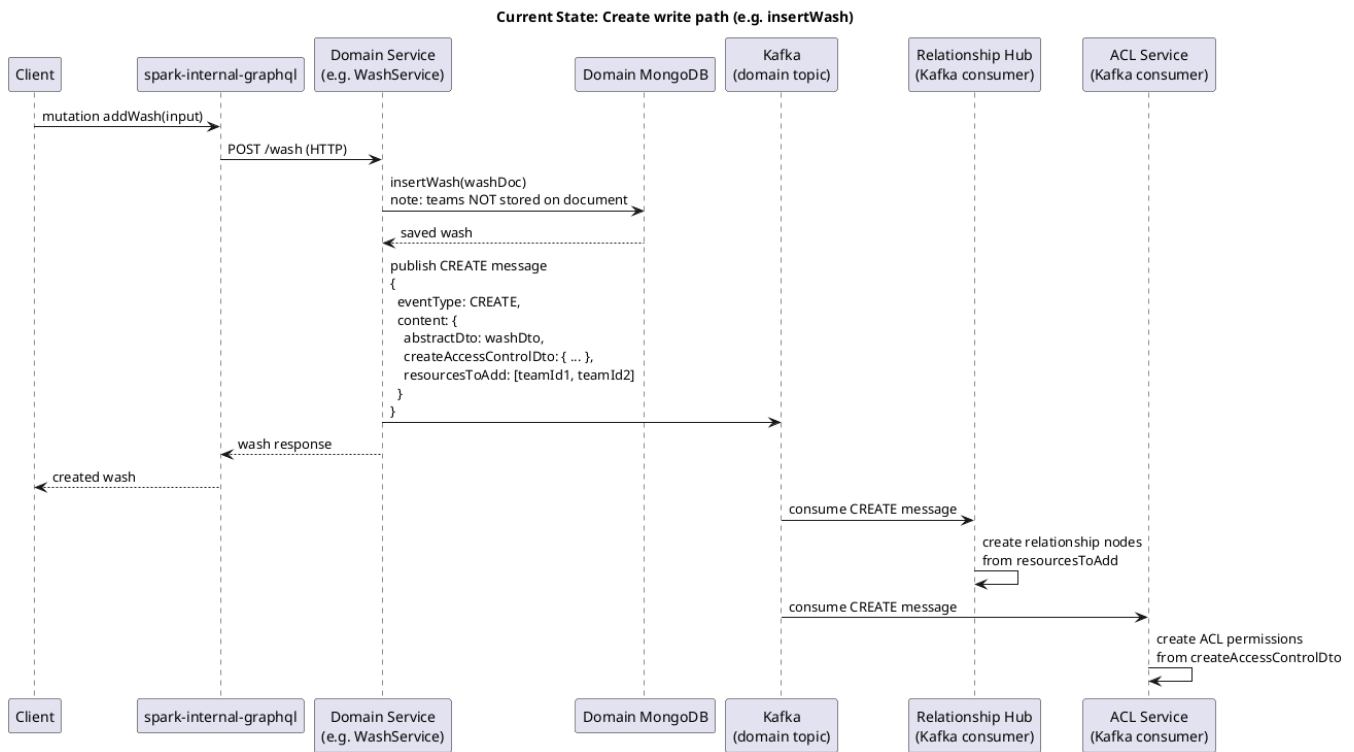
Measurement	getMeasurements	findRelationships type=measurement_set	Measurement service
FileLibrary	relatedPackagingComponents	whereUsedSearch	Packaging service

Current State — Read Flow

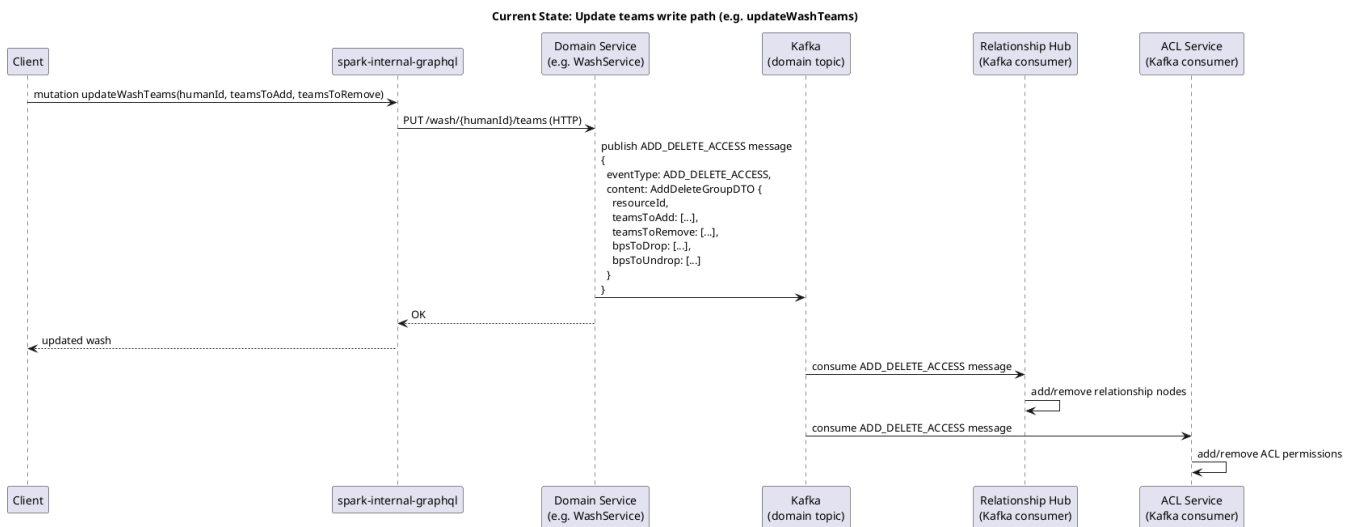


Current State — Write Flow (Create)

Domain services do **not** call the Relationship Service or ACL Service directly. All association writes are event-driven via Kafka. The domain service saves its entity to its own MongoDB and publishes a single Kafka message. Downstream consumers (ACL Service, Relationship Hub) process their respective blocks from that message.



Current State — Write Flow (Update Teams)



The Federation Gap

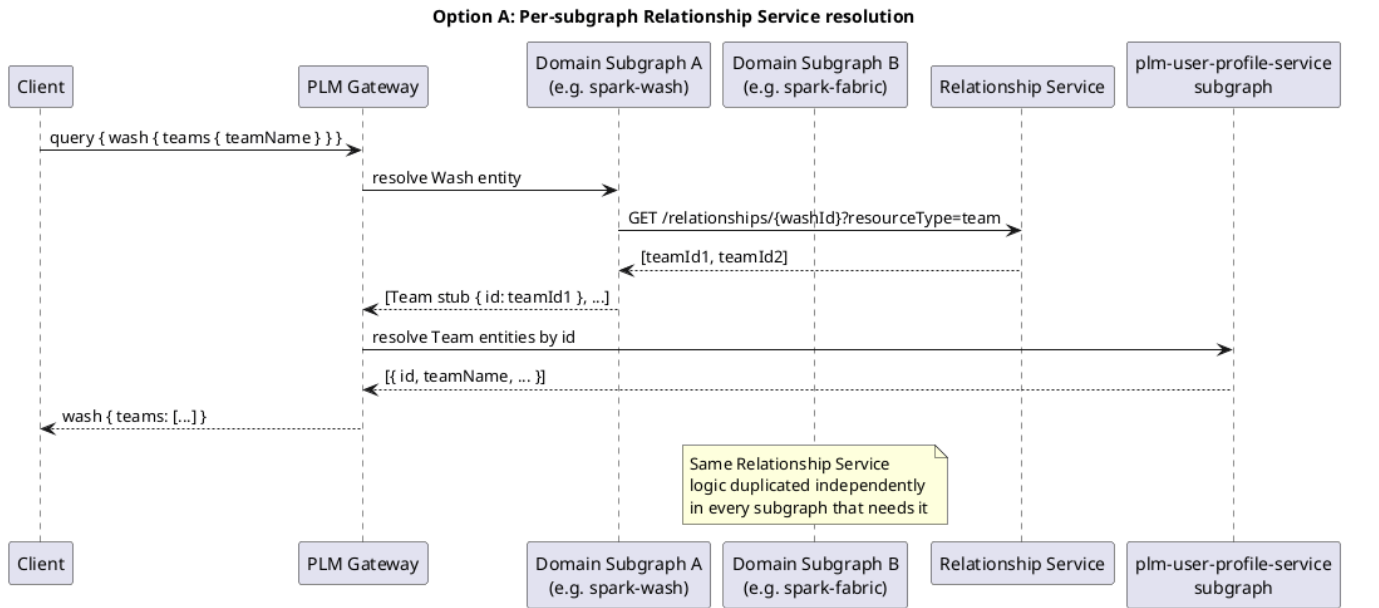
In the federated supergraph, each domain is a separate DGS subgraph. The shared resolver utilities from the monolith do not exist in the federated layer. A decision is needed on where the Relationship Service call logic lives when a federated subgraph needs to resolve one of the fields listed above.

Note: the **write path is already correct** — domain services publish Kafka messages that Relationship Hub and ACL Service consume. The gap is read-only: the federated subgraphs have no mechanism to query the Relationship Service at read time.

Options Considered

Option A — Each domain subgraph resolves Relationship Service fields independently

Each subgraph that needs to resolve a Relationship-backed field adds its own REST client for the Relationship Service and implements the lookup logic directly.



Pros:

- No new subgraph or shared infrastructure required
- Each domain team has full ownership and control of their resolver
- Straightforward to implement per domain in isolation

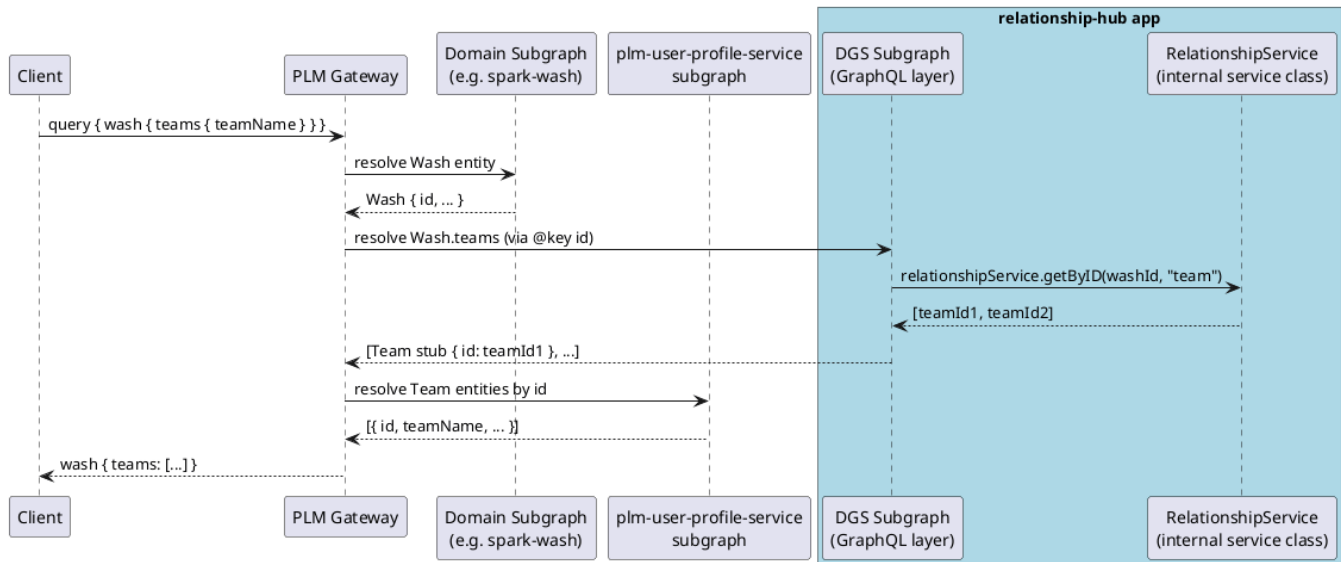
Cons:

- Resolver logic is spread across many subgraphs; while a shared library within the `plm-materials` monorepo could centralize utilities, each subgraph still requires its own wiring and configuration
- Changes to the Relationship Service API or auth pattern require updates across subgraphs; the concern grows for domains outside `plm-materials` (e.g. Product, Workspace, BOM in separate repos) — there is no shared enforcement point
- The Relationship Service is a cross-cutting platform concern used by Product, Workspace, Samples, Attachments, Discussions, and Measurements — fragmenting its call logic across many subgraphs increases the risk of inconsistent behavior

Option B — Dedicated relationship-hub subgraph extends all entity types

A new `relationship-hub` subgraph is added to the federation. It uses Apollo Federation `@key` to extend entity types across domains with their Relationship-backed fields.

Option B: relationship-hub subgraph (GraphQL layer + service in same app)



Pros:

- Relationship Service logic lives in one place
- A single subgraph to update if the Relationship Service API or auth pattern changes
- Mirrors the cross-cutting commonResolvers/ pattern from the monolith
- Acts as a single enforcement point — domains outside plm-materials (e.g. Product, BOM, Workspace in separate repos) could extend the same subgraph rather than re-implementing the pattern independently

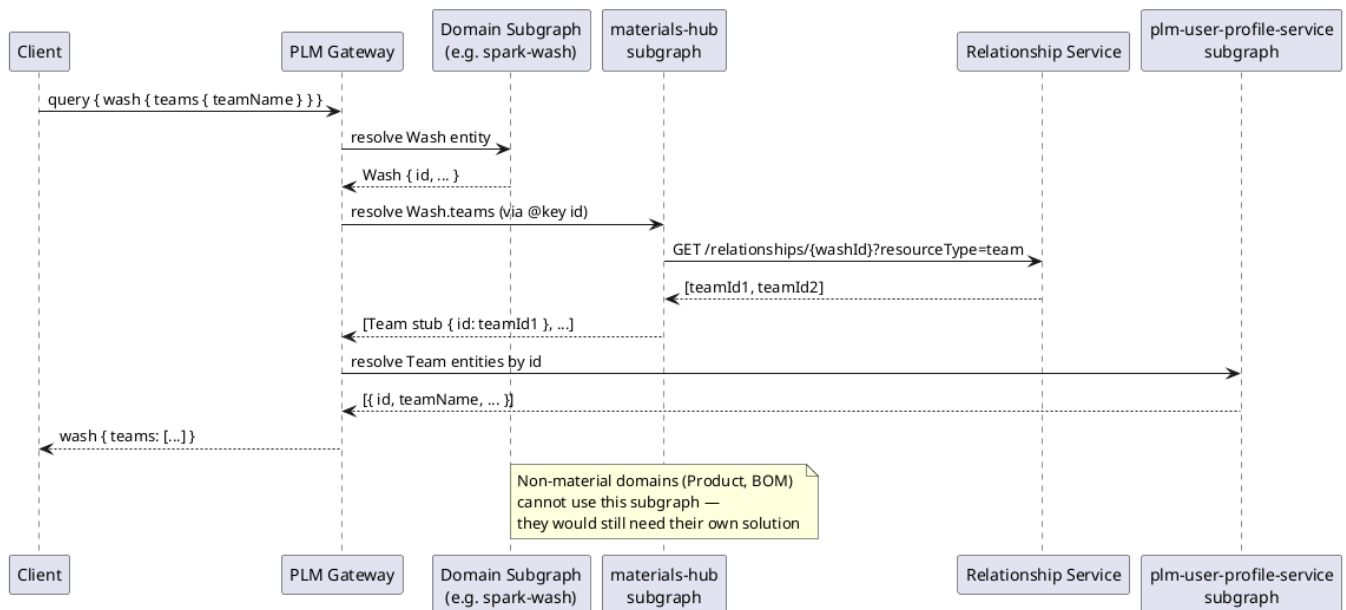
Cons:

- Requires adding a GraphQL transport layer (DGS subgraph) to the existing relationship-hub service
- As new entity types are onboarded to the federation from other repos (e.g. Product, BOM), relationship-hub must be updated to declare the corresponding entity stubs — a low-friction change but a coordination touch point across team boundaries

Option C — Existing materials-hub subgraph owns cross-cutting Relationship resolution

The existing materials-hub subgraph is extended to resolve Relationship-backed fields for all material entity types.

Option C: materials-hub owns Relationship resolution



Pros:

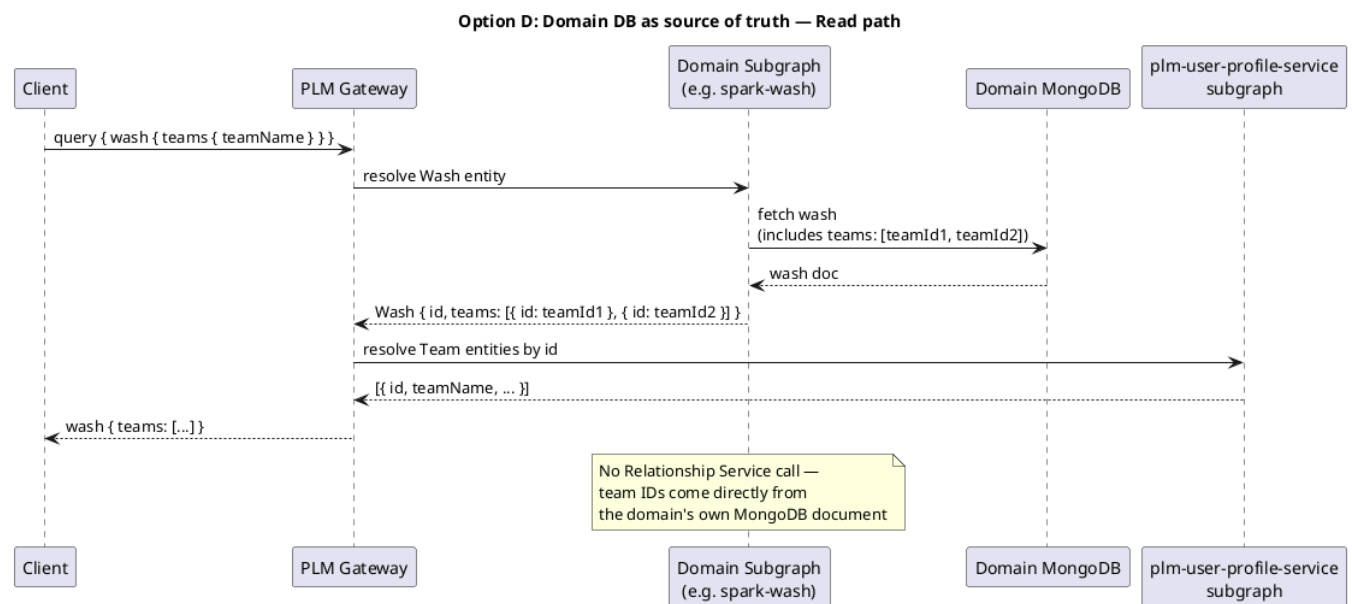
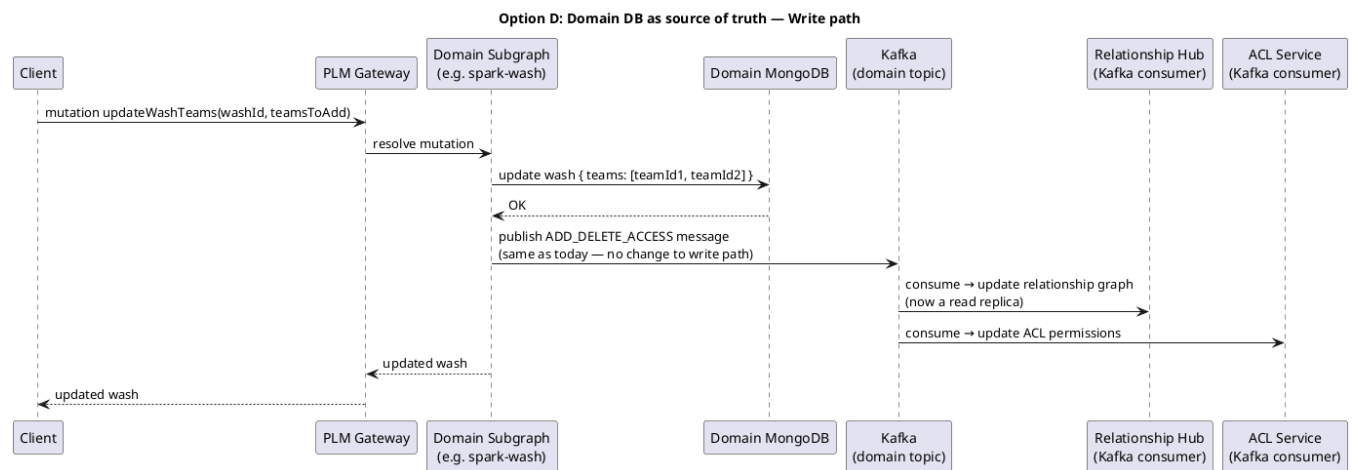
- No new subgraph required
- Reuses existing infrastructure and deployment pipeline

Cons:

- Conflates the Hub/BaseMaterial domain with a cross-cutting platform concern
- materials-hub ownership and scope becomes less clear
- Non-material domains (Product, BOM, Workspace) cannot use this subgraph — they would still need their own solution
- Changes to Relationship resolution logic are coupled to the Hub domain release cycle

Option D — Domain services become the source of truth; Relationship Service becomes a read replica

Each domain service stores associations directly on its own MongoDB document (e.g. `wash.teams: [teamId]`). The Relationship Service is retained as a read replica, kept in sync via the existing Kafka event stream. Domain subgraphs resolve association fields directly from their own data store — no Relationship Service call at read time.



Pros:

- Eliminates the Relationship Service call at read time — associations are co-located with the entity
- Write path is additive — domain service already publishes Kafka messages; the only change is also persisting team IDs to the domain document
- Existing Kafka consumers (Relationship Hub, ACL Service) are unaffected — Relationship Service remains available as a read replica
- Read cutover can be done incrementally per domain
- Aligns ownership: the domain service that owns the entity also owns its associations

Cons:

- Requires a one-time data migration to backfill existing associations into each domain's MongoDB document
- Increases the scope of each domain migration story — association persistence must be added to each domain service's write path
- The Relationship Service serves non-material consumers and cross-entity "where used" queries (e.g. Product, Workspace, BOM) — those use cases must be inventoried to ensure the read replica remains accurate
- Source-of-truth shift is a larger architectural change — requires broader team alignment before implementation can begin
- Does not address non-material domains (Product, BOM, Workspace) that also use the Relationship Service — they would still need a resolution strategy

Evaluation Summary

Criteria	Option A	Option B	Option C	Option D
Implementation effort	Low per domain, high total	Medium (one-time)	Medium	High (migration + per domain)
Maintenance surface	High (many subgraphs)	Low (1 subgraph)	Low (1 subgraph)	Low (per domain, co-located)
Mirrors monolith pattern	Partially	Yes	No	No — improves on it
Introduces new subgraph	No	No (adds GQL layer to existing service)	No	No
Domain ownership clarity	High	High	Low	High
Risk of behavioral divergence	High	Low	Low	Low
Cross-repo extensibility	Low	High	Low (materials only)	Low
Query-time performance	Same as today	Same as today	Same as today	Improved (no hop)
Requires data migration	No	No	No	Yes
Disruption to existing consumers	None	None	None	None (Kafka replica preserved)

Decision Outcome

Chosen option: Option D — Domain services become the source of truth; Relationship Service becomes a read replica

Rationale: The group decided that relationship-hub is more than likely not the future and we want to explore options that don't involve it. We'd ultimately like to sunset the relationship-hub application and let the hive gql federation provide relationships directly via the domain service or via entity extension pattern described [here](#) and [here](#).

Consequences

TBD — to be filled in after decision is made.

Follow-Up Work

- [] Confirm auth pattern for Relationship Service calls from DGS subgraphs (service-to-service token or pass-through)
- [] Inventory all non-PLM-materials consumers of the Relationship Service before any source-of-truth shift (relevant to Option D)
- [] Update all 8 material domain backend stories to reference this ADR for teams, files, connectedBoms, and other Relationship-backed fields
- [] Unblock **SPARK-30219** (Wash teams field)

References

- [SPARK-30228](#) — ADR Jira story
- [SPARK-30219](#) — Wash teams field (blocked)
- [Materials on GQL Federation - Generic Subgraph](#)
- `spark-internal-graphql/packages/data-source-spark/src/services/Relationship.js`
- `spark-internal-graphql/packages/data-source-spark/src/resolvers/commonResolvers/Teams.js`
- `plm-materials/apps/spark-wash/src/main/java/com/tgt/spark/service/WashService.java`
- `plm-materials/apps/spark-wash/src/main/java/com/tgt/spark/service/KafkaService.java`
- `plm-materials/apps/spark-wash/src/main/java/com/tgt/spark/dto/CreateWashKafkaMessageDTO.java`
- `plm-user-profile-service/src/main/resources/schema/team.graphqls`